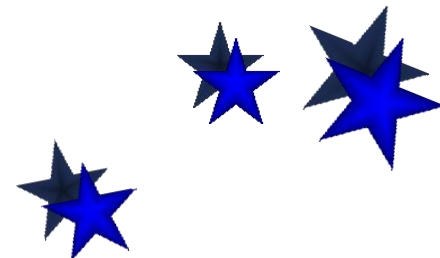


1.3 算法的数学基础

■ 有关函数渐近的界的定理

定理1.1 设 f 和 g 是定义域为自然数集合的函数.

- (1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0$, 那么 $f(n) = O(g(n))$ 。
- (2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty$, 那么 $f(n) = \Omega(g(n))$ 。
- (3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, 常数 $c > 0$, 那么 $f(n) = \Theta(g(n))$ 。
- (4) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, 那么 $f(n) = o(g(n))$ 。
- (5) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$, 那么 $f(n) = \omega(g(n))$ 。



1.3 算法的数学基础

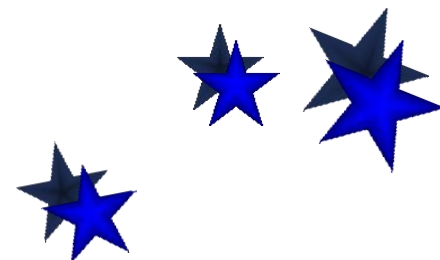
- 定理1.1的应用——估计函数的阶

例4 设 $f(n) = \frac{1}{2}n^2 - 3n$, 证明 $f(n) = \Theta(n^2)$.

证 因为

$$\lim_{n \rightarrow +\infty} \frac{f(n)}{n^2} = \lim_{n \rightarrow +\infty} \frac{\frac{1}{2}n^2 - 3n}{n^2} = \frac{1}{2}$$

根据定理1.1有 $f(n) = \Theta(n^2)$.



1.3 算法的数学基础

- 一些重要结论：可以证明

(1) 多项式函数的阶低于指数函数

$$n^d = o(r^n), \quad r > 1, \quad d > 0$$

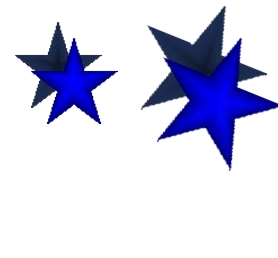
$$\lim_{n \rightarrow \infty} \frac{n^d}{r^n} = \lim_{n \rightarrow \infty} \frac{dn^{d-1}}{r^n \ln r} = \lim_{n \rightarrow \infty} \frac{d(d-1)n^{d-2}}{r^n (\ln r)^2} = \dots = \lim_{n \rightarrow \infty} \frac{d!}{r^n (\ln r)^d} = 0$$

分子分母
求导数

(2) 对数函数的阶低于幂函数的阶

$$\ln n = o(n^d), \quad d > 0$$

$$\lim_{n \rightarrow \infty} \frac{\ln n}{n^d} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{dn^{d-1}} = \lim_{n \rightarrow \infty} \frac{1}{dn^d} = 0$$



1.3 算法的数学基础

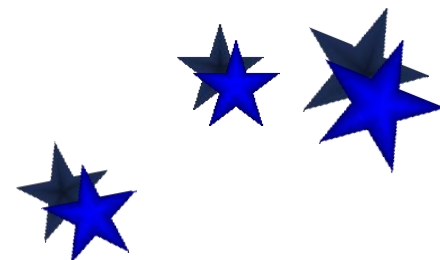
定理1.2 设 f, g, h 是定义域为自然数集合的函数,

(1) 如果 $f=O(g)$ 且 $g=O(h)$, 那么 $f=O(h)$.

(2) 如果 $f=\Omega(g)$ 且 $g=\Omega(h)$, 那么 $f=\Omega(h)$.

(3) 如果 $f=\Theta(g)$ 和 $g=\Theta(h)$, 那么 $f=\Theta(h)$.

定理1.2说明: 函数的阶之间的关系具有传递性



1.3 算法的数学基础

- 定理1.2的应用

例5：按照阶从高到低排序以下函数

$$f(n)=(n^2+n)/2, \quad g(n)=10n$$

$$h(n)=1.5^n, \quad t(n)=n^{1/2}$$

解：根据定理1.1，可给出如下函数关系

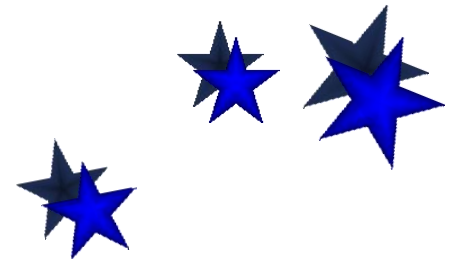
$$h(n) = \omega(f(n)),$$

$$f(n) = \omega(g(n)),$$

$$g(n) = \omega(t(n)),$$

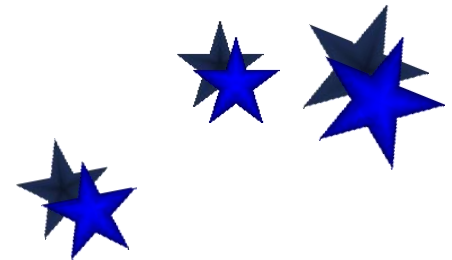
排序结果： $h(n), f(n), g(n), t(n)$

定理1.2 传递性



1.3 算法的数学基础

- 定理1.3 假设函数 f 和 g 的定义域为自然数集, 若对某个其它函数 h , 有 $f=O(h)$ 和 $g=O(h)$, 那么 $f+g=O(h)$
- 定理1.3可以推广到有限个函数: 算法由有限步骤构成. 若每一步的时间复杂度函数的上界都是 $h(n)$, 那么该算法的时间复杂度函数可以写作 $O(h(n))$.
- 算法的时间复杂度是各步操作时间之和, 在常数步的情况下取最高阶的函数即可.



1.3 算法的数学基础

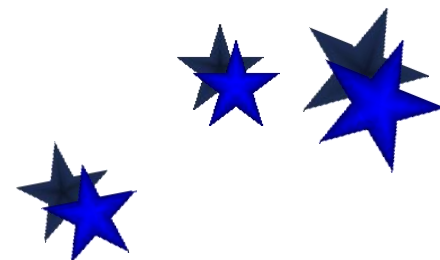
■ 几类重要的函数——自学

阶的高低

至少指数级: $2^n, 3^n, n!, \dots$

多项式级: $n, n^2, n \log n, n^{1/2}, \dots$

对数多项式级: $\log n, \log^2 n, \log \log n, \dots$



1.3 算法的数学基础

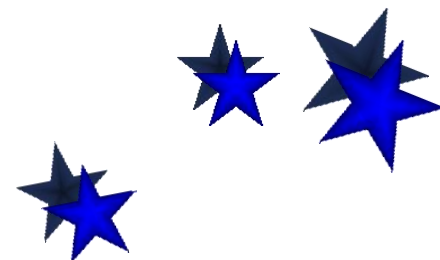
- 常见函数的阶从高到低的排序

$$2^{2^n}, n!, n2^n, (3/2)^n, (\log n)^{\log n} = n^{\log \log n},$$

$$n^3, \log(n!) = \Theta(n \log n), n = \Theta(2^{\log n}),$$

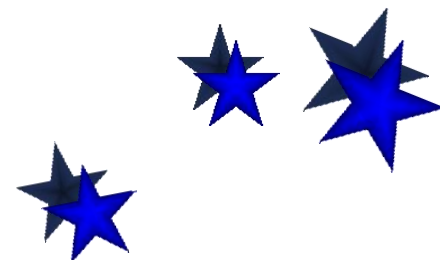
$$\log^2 n, \log n, \sqrt{\log n}, \log \log n,$$

$$n^{1/\log n} = \Theta(1)$$



1.3 算法的数学基础

- 小结
 - 几类常用函数的阶的性质
 - ✓ 对数函数
 - ✓ 指数函数
 - ✓ 阶乘函数
 - ✓ 取整函数
 - 如何利用上述性质估计函数的阶？



1.3 算法的数学基础

■ 算法复杂度分析方法

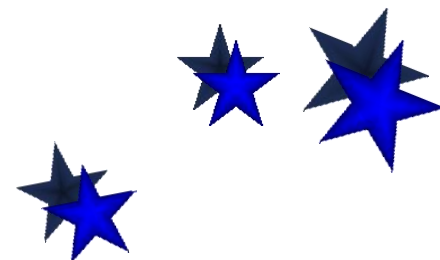
- 算法分类

- (1) 循环类（迭代）算法

- 该类算法的时间复杂度通常表现为算法中基本运算的执行次数的累加和。

- (2) 递归类算法

- 该类算法的时执行时间通常表示为递归方程形式，因此需要对递归方程进行求解，找出算法的时间复杂度表示。



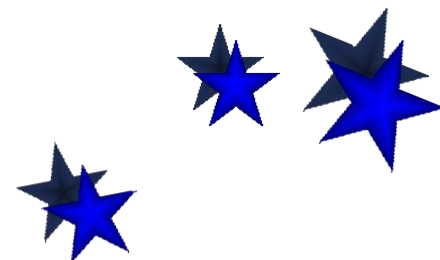
1.3 算法的数学基础

- 循环类算法时间复杂度分析
 - 循序次数直接依赖问题规模 n :

```
x=0;  
for(k=1;k<=n;k++)  
    x=x+1;  
    for(i=1;i<=n;i++)  
        for(j=1;j<=n;j++)  
            y=y+1;
```

最内层语句执行次数: n^2

时间复杂度: $O(n^2)$



1.3 算法的数学基础

- 循环次数间接依赖问题规模 n

$x=1;$

$for(i=1;i\leq n;i++)$

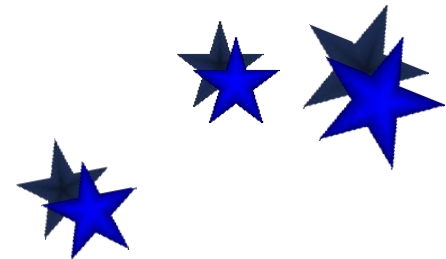
$for(j=1;j\leq i;j++)$

$for(k=1;k\leq j;k++)$

$x=x+1;$

$$\begin{aligned} 1 &= \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right) \\ &= \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right) \end{aligned}$$

时间复杂度: $O(n^3)$



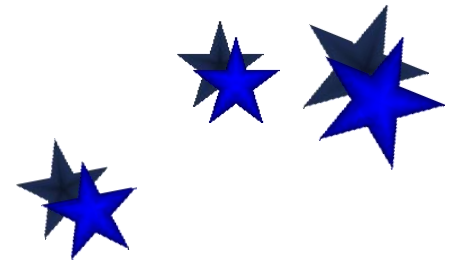
1.3 算法的数学基础

- 举例1：冒泡排序：数组中的n个元素由小到大进行排序

```
1. template <class T>  
2. void bubble(Type A[],int n)  
3. {  
4.     int i,k;  
5.     for (k=n-1;k>0;k--) {  
6.         for (i=0;i<k;i++)  
7.             if (A[i] > A[i+1]) {  
8.                 swap(A[i],A[i+1]);  
9.             }  
10.        }  
11.    }  
12. }
```

$$1 + 2 + 3 + \cdots + (n - 1) \\ = \frac{n(n - 1)}{2}$$

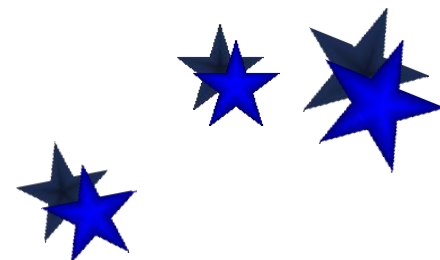
时间复杂度： $O(n^2)$



1.3 算法的数学基础

举例2：选手的竞技淘汰比赛——锦标赛排序

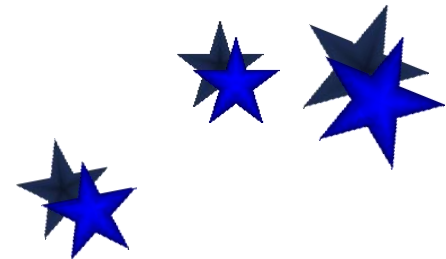
有 $n = 2^k$ 位选手进行竞技淘汰比赛，最后决出冠军的选手。假定函数：*int comp(Type mem1, Type mem2)* 模拟两位选手的比赛，若 *mem1* 胜则返回1，否则返回0。并假定可以在常数时间 c 内完成函数 *comp* 的执行。



1.3 算法的数学基础

```
1. Type game(Type group[],int n)
2. {
3.     int j,i = n;
4.     while (i>1) { /*n=2k 共进行k次循环*/
5.         i = i / 2;
6.         for (j=0;j<i;j++) /*每一轮需要进行i/2次比赛*/
7.             if (comp(group[j+i],group[j]))
8.                 group[j] = group[j+i];
9.     }
10.    return group[0];
11. }
```

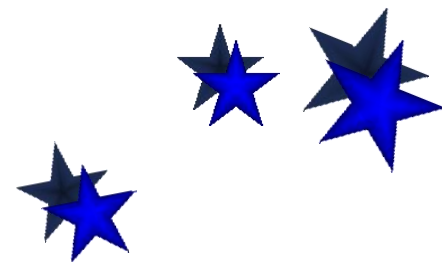
$$T(n) = \frac{n}{2} + \frac{n}{4} + \cdots + \frac{n}{2^k} = n \sum_{i=1}^k \frac{1}{2^i} \quad (k = \log n)$$



1.3 算法的数学基础

- 举例3：洗牌程序

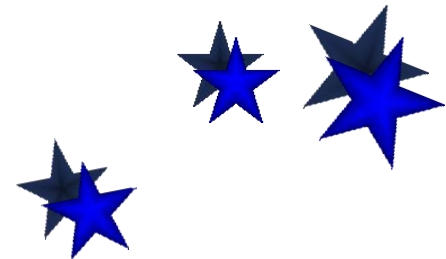
对 n 张牌进行 n 次洗牌，洗牌规则如下：在第 k 次洗牌时（ $k=1, \dots, n$ ），对第 i 张牌（ $i=1, \dots, n/k$ ）随机地产生一个小于 n 的正整数 d ，互换第 i 张牌和第 d 张牌的位置。



1.3 算法的数学基础

```
1. void shuffle(Type A[],int n)
2. {
3.     int i,k,m,d;
4.     random_seed(0);    /*初始化随机数的种子*/
5.     for (k=1;k<=n;k++) {
6.         m = n / k ;
7.         for (i=1;i<=m;i++) {
8.             d = random()%n; /*生成一个小于n的随机数*/
9.             swap(A[i],A[d]);
10.        }
11.    }
12. }
```

$$T(n) = \sum_{i=1}^n \frac{n}{i} = n \sum_{i=1}^n \frac{1}{i}$$



1.3 算法的数学基础

- 举例4：二分检索算法（假设 $n=2^k-1$ ）

算法 **BinarySearch** (T, l, r, x)

输入：数组 T ，下标从 l 到 r ；数 x

输出： j

1. $l \leftarrow 1; r \leftarrow n$

2. **while** $l \leq r$ **do**

3. $m \leftarrow \lfloor (l+r)/2 \rfloor$

4. **if** $T[m]=x$ **then return** m // x 中位元素

5. **else if** $T[m] > x$ **then** $r \leftarrow m-1$

- 实例个数： $2n+1$

- 查找概率： $p_i=1/(2n+1)$

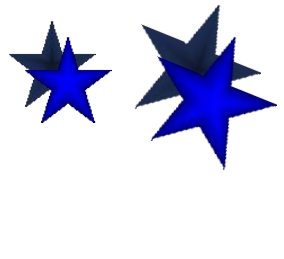
- 查找成功比较次数

$$\sum_{i=1}^k i 2^{i-1}$$

- 查找不成功比较次数：

$$k(n+1)$$

$$T(n) = \frac{1}{2n+1} \left(\sum_{i=1}^k i 2^{i-1} + k(n+1) \right)$$



1.3 算法的数学基础

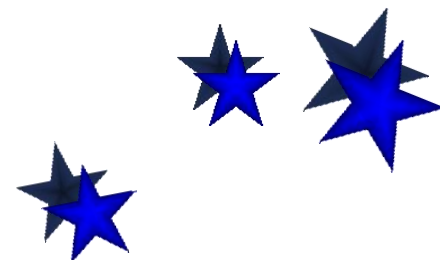
- 求和的方法

- (1) 等差、等比数列和调合级数求和

$$\sum_{k=1}^n a_k = \frac{n(a_1 + a_n)}{2}$$

$$\sum_{k=0}^n aq^k = \frac{a(1-q^{n+1})}{1-q}, \quad \sum_{k=0}^{\infty} aq^k = \frac{a}{1-q} \quad (q < 1)$$

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1) \quad \text{调合级数}$$



1.3 算法的数学基础

(2) 某些求和公式，求不出精确的值，可以估计和的上界

- 具体方法：使用放大法，使数列中的某些项放大，使数列变成一个类似于等比或等差数列的形式，然后再求和。
- 放大方法1：用序列中的最大项替代每一项

$$T(n) = \sum_{k=1}^n a_k \leq na_{\max}$$

优点：简单

缺点：放大后求得的和可能与原数列的和差距过大。但如果放大后所求得和函数的阶与原公式的阶保持不变，这种放大还是有意义的。



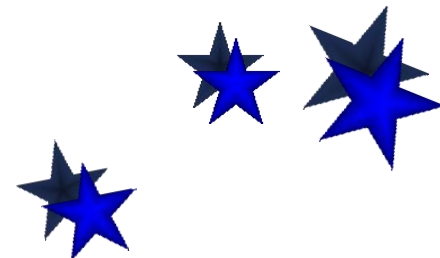
1.3 算法的数学基础

- 方大方法2：等比级数

假设存在常数 $r < 1$ ，使得 对一切 $k \geq 0$ 有 $a_{k+1}/a_k \leq r$ 成立。

$$T(n) = \sum_{k=0}^n a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1-r}$$

$$a_1 \leq a_0 r, a_2 \leq a_1 r \leq a_0 r^2, \dots$$



1.3 算法的数学基础

例7 估计 $\sum_{k=1}^n \frac{k}{3^k}$ 的上界.

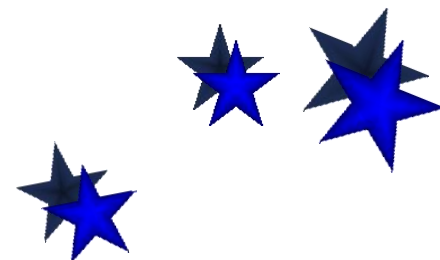
解:

$$a_k = \frac{k}{3^k}, \quad a_{k+1} = \frac{k+1}{3^{k+1}}$$

得

$$r = \frac{a_{k+1}}{a_k} = \frac{1}{3} \frac{k+1}{k} \leq \frac{2}{3}$$

$$\sum_{k=1}^n \frac{k}{3^k} \leq \sum_{k=1}^{\infty} \frac{1}{3} \left(\frac{2}{3}\right)^{k-1} = \frac{1}{3} \frac{1}{1 - \frac{2}{3}} = 1$$

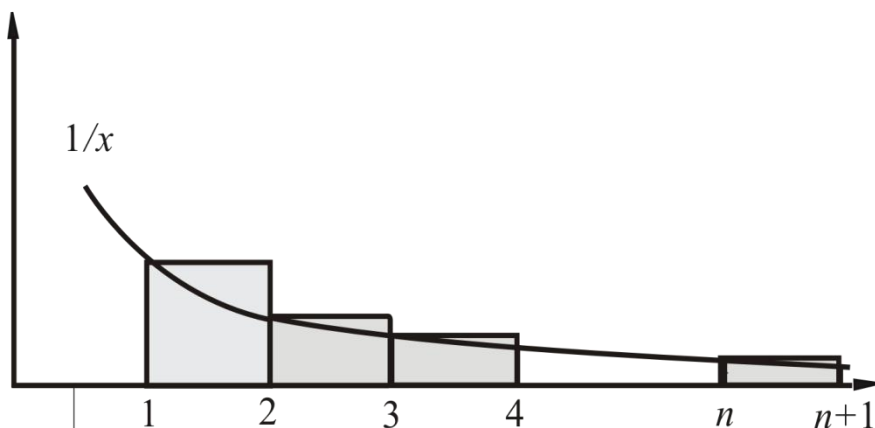


1.3 算法的数学基础

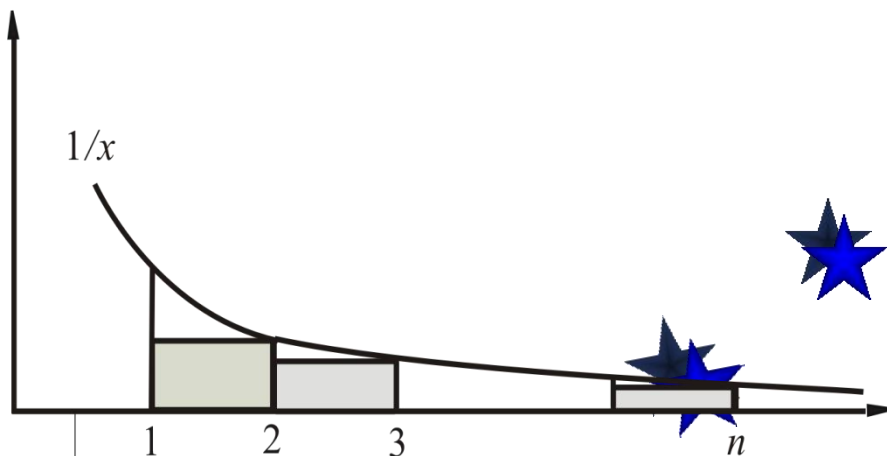
- 方大方法3：利用积分求出累加和的渐进的界——调合级数

估计 $\sum_{k=1}^n \frac{1}{k}$ 的渐近的界.

$$\begin{aligned}\sum_{k=1}^n \frac{1}{k} &\geq \int_1^{n+1} \frac{dx}{x} \\ &= \ln(n+1)\end{aligned}$$



$$\begin{aligned}\sum_{k=1}^n \frac{1}{k} &= 1 + \sum_{k=2}^n \frac{1}{k} \\ &\leq 1 + \int_1^n \frac{dx}{x} \\ &= \ln n + 1\end{aligned}$$



1.3 算法的数学基础

- 递归算法时间复杂度分析
 - 递归算法的时间复杂度也通常以递归的形式表示

举例1：求 $n!$

算法1 **fac** (n)

输入：整数 n

输出： $n!$

1. $f \leftarrow 1; i \leftarrow 1$

2. while $i \leq n$ do

3. $f \leftarrow f * i$

4. return f $O(n)$

算法2 **fac** (n)

输入： 整数 n

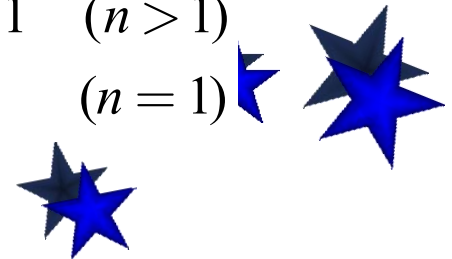
输出： $n!$

1. if $n=1$

2. then return 1

3. else return($n * \text{fac}(n-1)$)

$$F(n) = \begin{cases} F(n-1) + 1 & (n > 1) \\ 1 & (n = 1) \end{cases}$$



1.3 算法的数学基础

举例2: Hanoi问题

算法 **Hanoi** (A, C, n) // n 个盘子 A 到 C

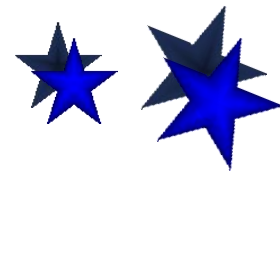
1. if $n=1$ then move (A, C) // 1个盘子 A 到 C

2. else Hanoi ($A, B, n-1$)

3. move (A, C)

4. Hanoi ($B, C, n-1$)

$$F(n) = \begin{cases} 2F(n-1) + 1 & (n > 1) \\ 1 & (n = 1) \end{cases}$$



1.3 算法的数学基础

举例3：直接插入排序

算法1 InsertSort(A, n)

1. for $j \leftarrow 2$ to n do
2. $x \leftarrow A[j]$
3. $i \leftarrow j-1$
4. while $i > 0$ and $x < A[i]$ do
5. $A[i+1] \leftarrow A[i]$
6. $i \leftarrow i-1$
7. $A[i+1] \leftarrow x$

$$\text{最好: } F(n) = \begin{cases} F(n-1) + 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

算法2 InsertSort(A, n)

1. if $n > 1$
2. then InsertSort($A, n-1$)
3. $x \leftarrow A[n]$
3. $i \leftarrow n-1$
4. while $i > 0$ and $x < A[i]$ do
5. $A[i+1] \leftarrow A[i]$
6. $i \leftarrow i-1$
7. $A[i+1] \leftarrow x$

$$\text{最坏: } F(n) = \begin{cases} F(n-1) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

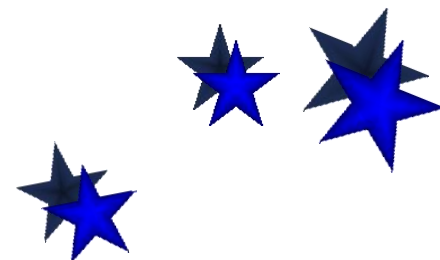


1.3 算法的数学基础

举例4：二路归并排序

算法 MergeSort(A, p, r) // 归并排序数组 $A[p..r]$

1. if $p < r$
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3. MergeSort(A, p, q)
4. MergeSort($A, q+1, r$)
5. Merge(A, p, q, r)



1.3 算法的数学基础

算法1.6 Merge(A, p, q, r) // 将排序数组 $A[p..q]$ 与 $A[q+1, r]$ 合并

1. $x \leftarrow q - p + 1, y \leftarrow r - q$ // x, y 分别为两个子数组的元素数

2. 将 $A[p..q]$ 复制到 $B[1..x]$, 将 $A[q+1..r]$ 复制到 $C[1..y]$

3. $i \leftarrow 1, j \leftarrow 1, k \leftarrow p$

4. While $i \leq x$ and $j \leq y$ do

5. if $B[i] \leq C[j]$ // B 的首元素不大于 C 的首元素

6. then $A[k] \leftarrow B[i]$ // 将 B 的首元素放到 A 中

7. $i \leftarrow i + 1$

8. else

9. $A[k] \leftarrow C[j]$

10. $j \leftarrow j + 1$

11. $k \leftarrow k + 1$

12. if $i > x$ then 将 $C[j..y]$ 复制到 $A[k..r]$ // B 已经是空数组

13. else 将 $B[i..x]$ 复制到 $A[k..r]$ // C 已经是空数组

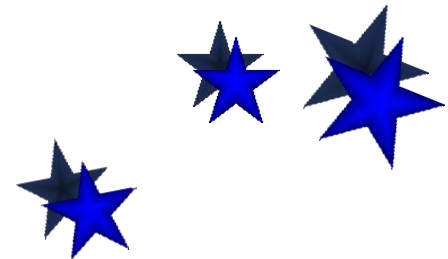
1.3 算法的数学基础

最好情况：

$$F(n) = \begin{cases} 2F(n/2) + n/2 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

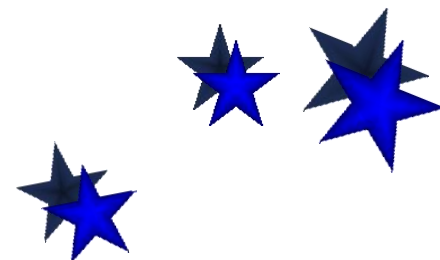
最坏情况：

$$F(n) = \begin{cases} 2F(n/2) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$



1.3 算法的数学基础

- 求解递归方程的方法
 - (1) 迭代法（递推法）
 - (2) 生成函数求解
 - (3) 特征方程求解
 - (4) 递归树
 - (5) 主方法

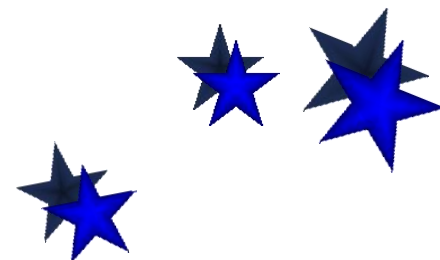


1.3 算法的数学基础

- 迭代法（递推法）求解递归方程

- 直接从递归方程出发，一层一层地往前递推，直到最前面的初始条件为止，就得到了问题的解。

$$(1)F(n) = \begin{cases} F(n-1) + 1 & (n > 1) \\ 1 & (n = 1) \end{cases}$$

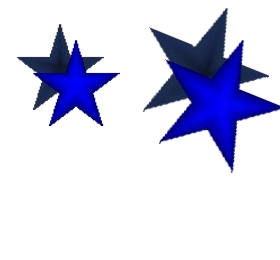


1.3 算法的数学基础

$$(2)F(n) = \begin{cases} 2F(n-1) + 1 & (n > 1) \\ 1 & (n = 1) \end{cases}$$

$$\begin{aligned} T(n) &= 2 \, \underline{T(n-1)} + 1 \\ &= 2 \, [\, \underline{2T(n-2)} + 1 \,] + 1 \\ &= 2^2 T(n-2) + 2 + 1 \\ &= \dots \\ &= 2^{n-1} T(1) + 2^{n-2} + 2^{n-3} + \dots + 2 + 1 \\ &= 2^{n-1} + 2^{n-1} - 1 \\ &= 2^n - 1 \end{aligned}$$

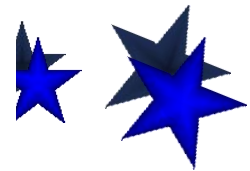
代入初值
求和



1.3 算法的数学基础

$$(3) F(n) = \begin{cases} F(n-1) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

$$\begin{aligned} W(n) &= W(n-1) + n-1 \\ &= [W(n-2) + n-2] + n-1 \\ &= W(n-2) + n-2 + n-1 \\ &= \dots \\ &= W(1) + 1 + 2 + \dots + (n-2) + (n-1) \\ &= 1 + 2 + \dots + (n-2) + (n-1) \\ &= n(n-1)/2 \end{aligned}$$

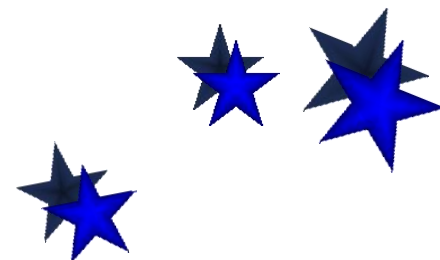


1.3 算法的数学基础

$$(4) F(n) = \begin{cases} 2F(n/2) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

□ 换元迭代

- 将对 n 的递推式换成对其他变元 k 的递推式
- 对 k 直接迭代
- 将解 (关于 k 的函数) 转换成关于 n 的函数



1.3 算法的数学基础

$$(4) F(n) = \begin{cases} 2F(n/2) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

假设 $n=2^k$, 递推方程如下:

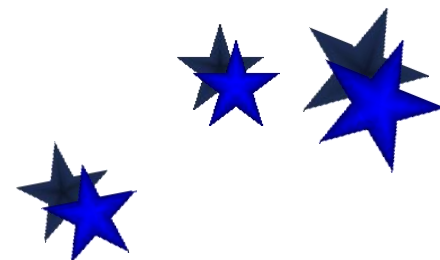
$$F(n) = 2F(n/2) + n - 1 \quad (n > 1)$$

$$F(1) = 0 \quad (n = 1)$$

换元:

$$F(2^k) = 2F(2^{k-1}) + 2^k - 1$$

$$F(0) = 0$$



1.3 算法的数学基础

$$(4) F(n) = \begin{cases} 2F(n/2) + n - 1 & (n > 1) \\ 0 & (n = 1) \end{cases}$$

$$W(n) = 2W(2^{k-1}) + 2^k - 1$$

$$= 2[2W(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1$$

$$= 2^2 W(2^{k-2}) + 2^k - 2 + 2^k - 1$$

$$= 2^2 [2W(2^{k-3}) + 2^{k-2} - 1] + 2^k - 2 + 2^k - 1$$

$$= \dots$$

$$= 2^k W(1) + k2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1)$$

$$= k2^k - 2^k + 1$$

$$= n \log n - n + 1$$

换元，
对k迭代



1.3 算法的数学基础

解的正确性——归纳法验证

证明:下述递推方程的解是 $W(n)=n(n-1)/2$

$$W(n)=W(n-1)+n-1$$

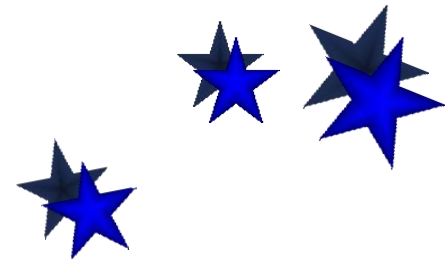
$$W(1)=0$$

方法: 数学归纳法

证 $n=1$, $W(1)=1*(1-1)/2 = 0$,

假设对于 n , 解满足方程, 则

$$\begin{aligned} W(n) &= W(n-1)+n-1 = (n-2)(n-1)/2 + n-1 \\ &= (n^2-n)/2 = n(n-1)/2 \end{aligned}$$



1.3 算法的数学基础

证明:下述递推方程的解是 $W(n)=n\log n-n+1$

$$W(n)=2W(n/2)+n-1$$

$$W(1)=0$$

方法: 数学归纳法

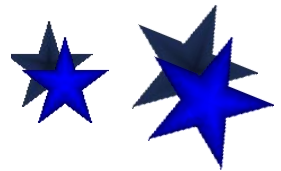
证 $n=1$, $W(1)=1*\log 1-1+1=0$,

假设对于 n , 解满足方程, 则

$$W(n) = 2W(n/2)+n-1 = 2\left(\frac{n}{2}\log\frac{n}{2}-\frac{n}{2}+1\right)+n-1$$

$$= n\log\frac{n}{2}-n+2+n-1$$

$$= n(\log n - 1) + 1 = n\log n - n + 1$$



1.3 算法的数学基础

快速排序平均时间分析

$$\begin{cases} T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + cn, & n \geq 2 \\ T(1) = 0 \end{cases} \quad \text{差消化简后迭代}$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2$$

$$nT(n) = (n+1)T(n-1) + 2cn - c$$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2cn - c}{n(n+1)} = \dots$$

$$= 2c \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} + \frac{T(1)}{2} \right] - O\left(\frac{1}{n}\right) = \Theta(\log n)$$

$$T(n) = \Theta(n \log n)$$

1.3 算法的数学基础

- 递归树求解递归方程

$T(n)=aT(n/b)+f(n)$ 分治策略

- ◆ 递归树的生成规则

- 初始，递归树只有根结点，其值为 $W(n)$

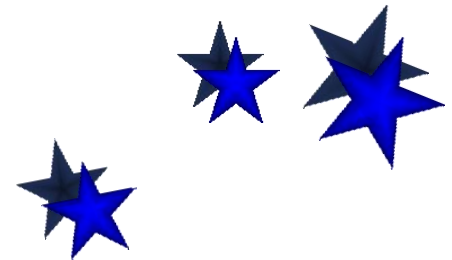
- 不断继续下述过程：

将函数项叶结点的迭代式 $W(m)$ 表示成二层子树

用该子树替换该叶结点

- 继续递归树的生成，直到树中无函数项 (只有初值)为止

.

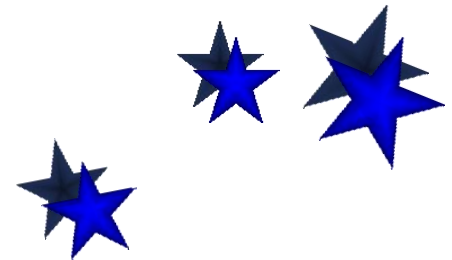
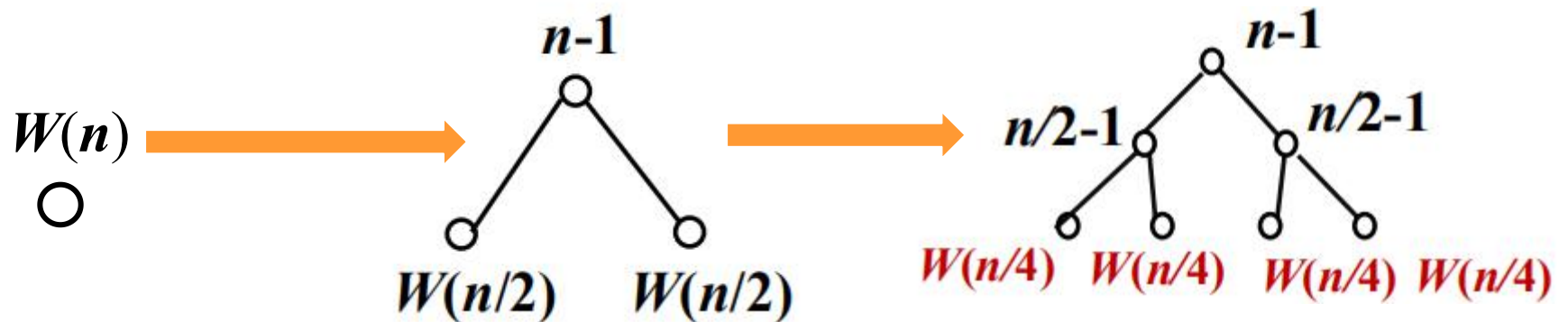


1.3 算法的数学基础

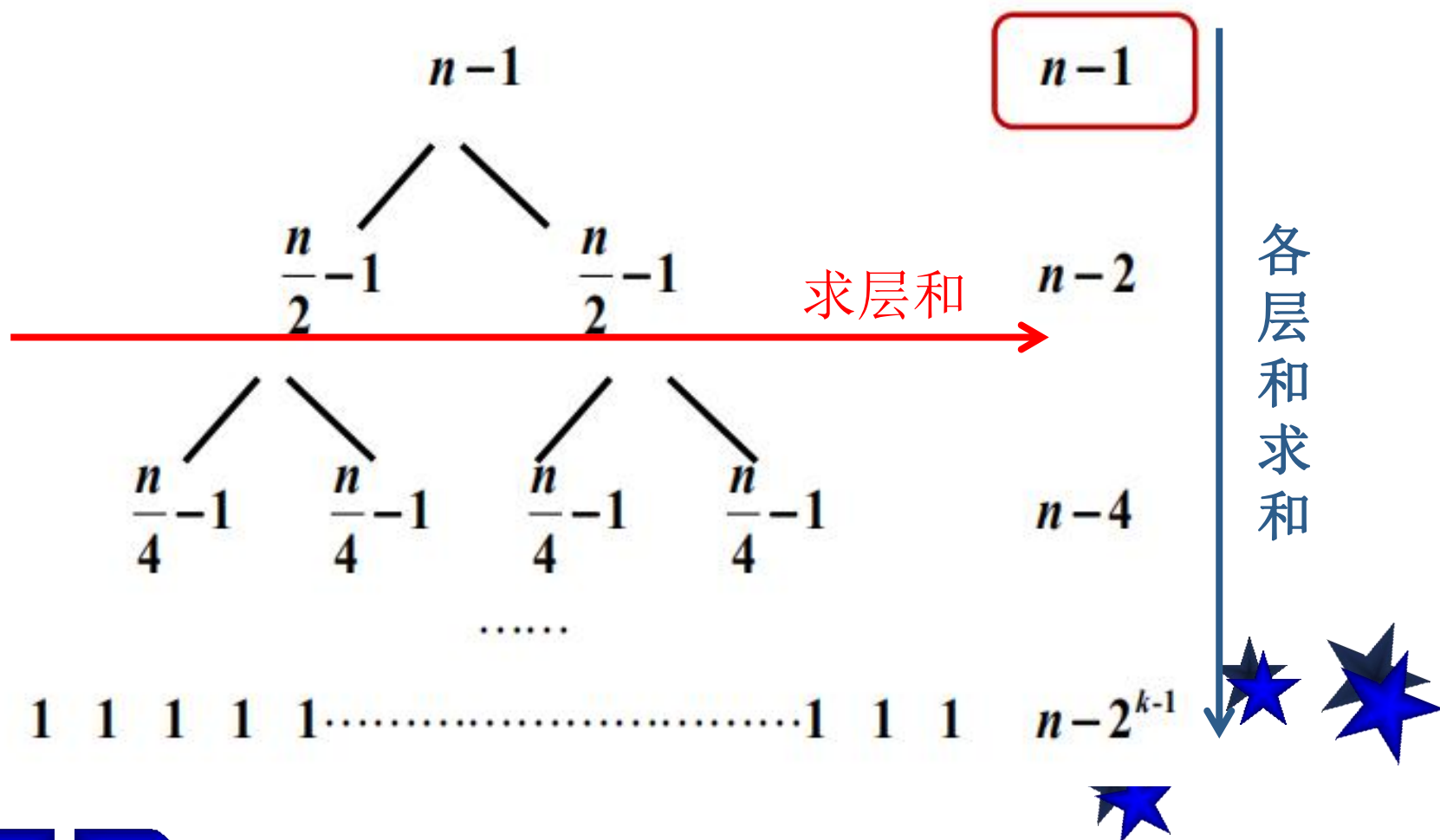
- 递归树求解递归方程

二分归并排序

$$W(n) = 2W(n/2) + \underline{n-1}$$



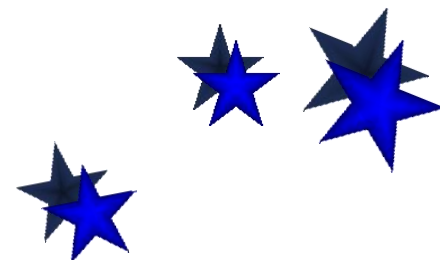
1.3 算法的数学基础



1.3 算法的数学基础

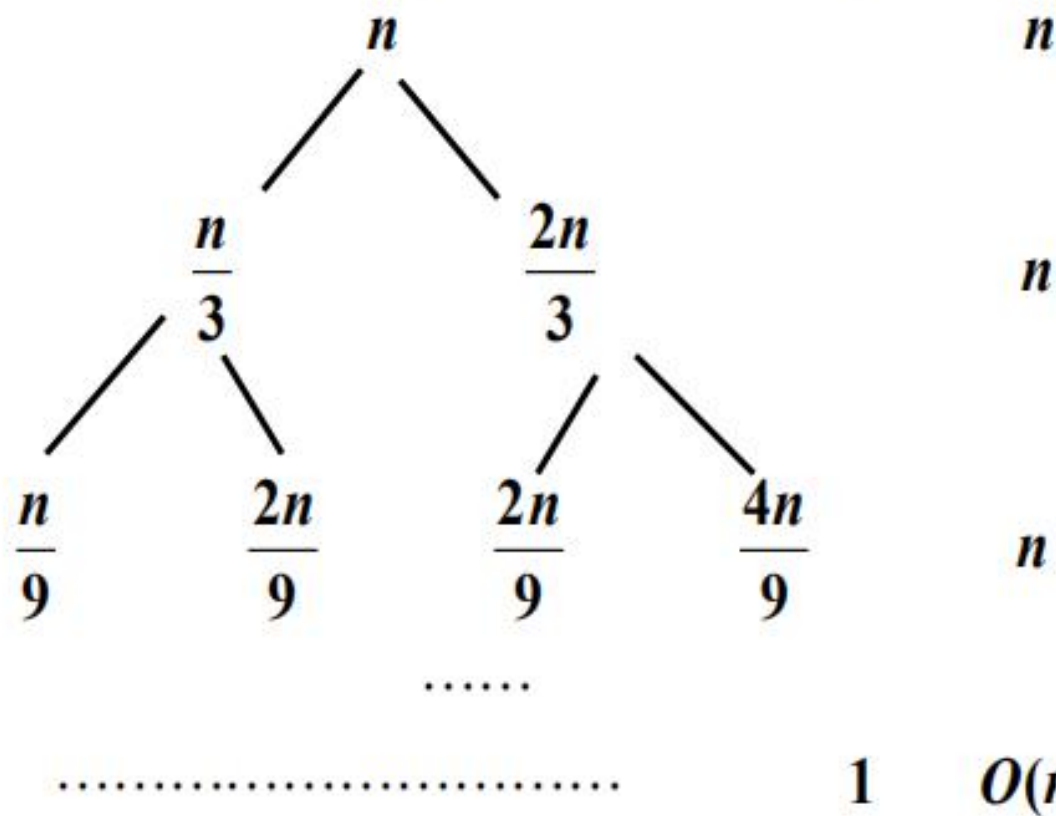
$$\begin{aligned} W(n) &= 2W(n/2) + n - 1, \quad n = 2^k, \\ W(1) &= 0 \end{aligned}$$

$$\begin{aligned} W(n) &= n - 1 + n - 2 + \dots + n - 2^{k-1} \\ &= kn - (2^k - 1) \\ &= n \log n - n + 1 \end{aligned}$$



1.3 算法的数学基础

求解方程: $T(n) = T(n/3) + T(2n/3) + n$



1.3 算法的数学基础

可见每层的值都为 n ，从根到叶节点的最长路径是：

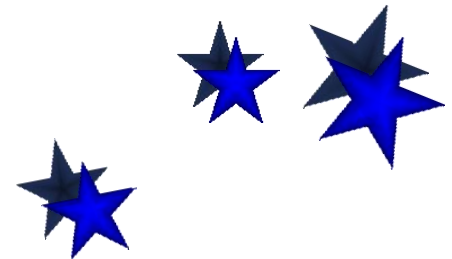
$$n \rightarrow \frac{2}{3}n \rightarrow \left(\frac{2}{3}\right)^2 n \rightarrow \dots \rightarrow 1$$

因为最后递归的停止是在 $(2/3)^k n = 1$. 则

$$k = \log_{3/2} n$$

$$T(n) \leq \sum_{i=0}^k n = (k+1)n = n(\log_{3/2} n + 1)$$

即 $T(n) = O(n \log n)$



1.3 算法的数学基础

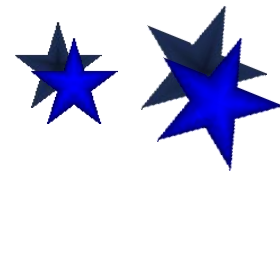
● 主定理（Master Method）求解递归方程

定理：设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为非负整数，且

$$T(n) = aT(n/b) + f(n) \text{ —分治策略}$$

则有以下结果：

1. 若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$
3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对某个常数 $c < 1$ 和所有充分大的 n 有 $a f(n/b) \leq c f(n)$, 那么
$$T(n) = \Theta(f(n))$$



1.3 算法的数学基础

例9 求解递推方程 $T(n) = 9T(n/3) + n$

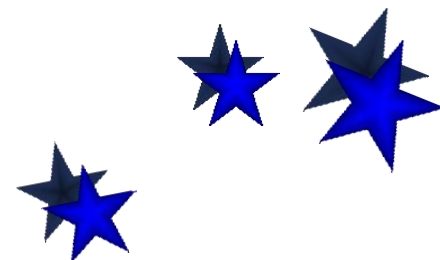
解 上述递推方程中的 $a = 9, b = 3, f(n) = n$, 那么

$$n^{\log_3 9} = n^2, \quad f(n) = O(n^{\log_3 9 - 1}),$$

相当于主定理的第一种情况, 其中 $\epsilon = 1$. 根据定理得到

$$T(n) = \Theta(n^2)$$

练习: $T(n) = 5T(n/2) + \Theta(n^2)$



1.3 算法的数学基础

例10 求解递推方程 $T(n) = T(2n/3) + 1$

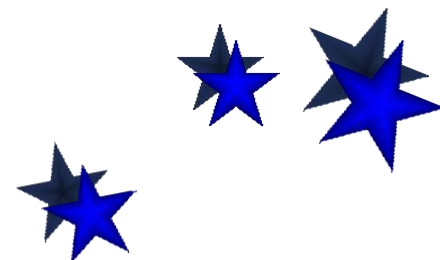
解 上述递推方程中的 $a=1, b=3/2, f(n)=1$, 那么

$$n^{\log_{3/2} 1} = n^0 = 1, \quad f(n) = 1$$

相当于主定理的第二种情况. 根据定理得到 .

$$T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

练习 $T(n) = 2T(n/2) + \Theta(n)$



1.3 算法的数学基础

练习: $T(n)=5T(n/2)+\Theta(n^3)$

例11 求解递推方程

$$T(n) = 3T(n/4) + n \log n$$

解 上述递推方程中的 $a=3, b=4, f(n)=n \log n$, 那么

$$n \log n = \Omega(n^{\log_4 3 + \varepsilon}) = \Omega(n^{0.793 + \varepsilon}),$$

$\varepsilon \approx 0.2$

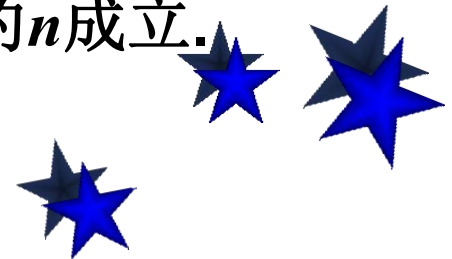
此外, 要使 $a f(n/b) \leq c f(n)$ 成立, 代入 $f(n)=n \log n$, 得到

$$\frac{3n}{4} \log \frac{n}{4} \leq c n \log n$$

显然只要 $c \geq 3/4$, 上述不等式就可以对充分大的 n 成立.

相当于主定理的第三种情况. 因此有

$$T(n) = \Theta(f(n)) = \Theta(n \log n)$$



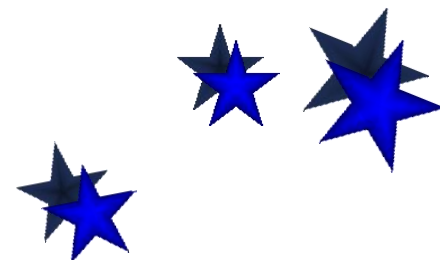
1.3 算法的数学基础

不能使用主定理的例子

例12 求解 $T(n) = 2T(n/2) + n \log n$

$a=b=2$, $n^{\log_2 2} = n$, $f(n) = n \log n$

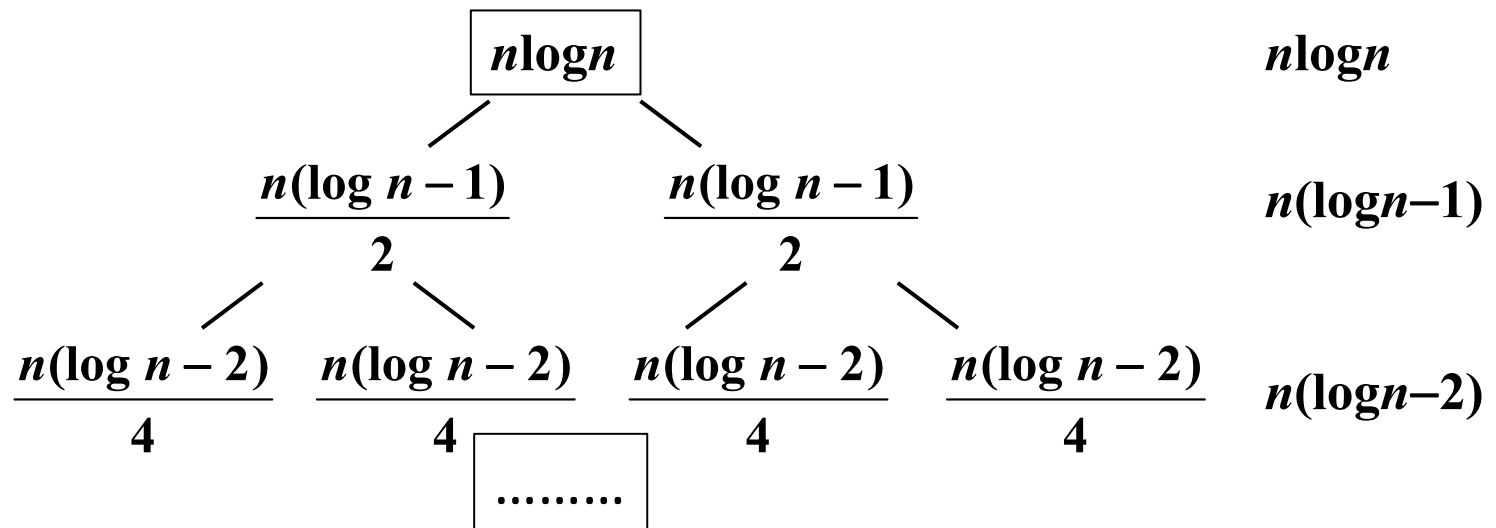
不存在 $\varepsilon > 0$ 使得 $n \log n = \Omega(n^{1+\varepsilon})$ 成立,



1.3 算法的数学基础

$$T(n) = 2T(n/2) + n \log n$$

$$n^{\log_2 2} = n$$



$$\begin{aligned} T(n) &= n \log n + n(\log n - 1) + n(\log n - 2) + \dots + n(\log n - k + 1) \\ &= (n \log n) \log n - n(1 + 2 + \dots + k - 1) \\ &= n \log^2 n - nk(k-1)/2 = O(n \log^2 n) \end{aligned}$$

